

Module 9 — Multi-Agent Systems

Roles, coordination, and shared context across cooperating agents

🎯 Learning goals

- Decide **when** multi-agent designs are worth the complexity.
- Model **roles**: router, researcher, critic, supervisor, executor.
- Implement **coordination patterns**: supervisor, swarm, debate, consensus.
- Share **context and tools** safely across agents.
- Apply **consensus** and conflict resolution strategies.
- **Spawn subagents** in parallel and reconcile what they send back.

When to go multi-agent

- **Yes**: different expertise needed (finance vs legal vs technical).
- **Yes**: critic/reviewer improves quality of high-stakes answers.
- **Yes**: parallel research on independent sub-questions.
- **Yes**: Context window management is needed.
- **Yes**: reduce costs by giving simple tasks to smaller agents.
- **No**: single-domain Q&A where one prompt + tools suffices.
- **No**: latency-sensitive paths where an extra round-trip hurts.

Xi et al. (2023). "The Rise and Potential of Large Language Model Based Agents: A Survey." arXiv:2309.07864

Common agent roles

- **Router**: classifies the query and dispatches to the right specialist.
- **Specialist**: domain expert with a focused system prompt and tools.
- **Critic**: reviews another agent's output for accuracy and gaps.
- **Supervisor**: orchestrates the team, merges results, decides when done.
- **Executor**: takes the final plan and runs it (deploy, notify, write).
- **Planner**: writes the plan based on the query and the context.
- **Conflict Resolver**: fixes conflicting results from the agents.
- **Reporter**: reports the final answer to the human user.

Li et al. (2023). "CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society." arXiv:2303.17760

Router + specialist agents

```
SPECIALISTS = {
    "finance": fin_agent,
    "technical": tech_agent,
    "legal": legal_agent,
}

def classify(query, client):
    """LLM returns JSON: {"department": "..."}"""
    ...

def route(query, client):
    dept = classify(query, client)
    return SPECIALISTS[dept].run(query, client)

# "What is our budget?" → finance
# "Why is the API slow?" → technical
# "Review the contract terms" → legal
```

- → LLM-based classification with structured outputs — a Literal schema means only valid departments come back
- → Each specialist has its own system prompt and domain focus



Demo — Specialist agents + router

Switch to terminal: python demo/demo.py — Part 1



Coordination patterns

Supervisor, debate, and other ways to organise cooperating agents

Coordination patterns

- **Supervisor**: one lead agent delegates tasks and merges results.
- **Swarm**: agents self-organise, passing context peer-to-peer.
- **Debate**: two agents argue opposing views; a judge decides.
- **Spawn**: the lead creates subagents on demand and merges their reports.
- **Blackboard**: shared state that agents read and write to.
- **Task queue**: agents pull work items from a shared queue.

Wu et al. (2023). "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." arXiv:2308.08155

Supervisor-critic pipeline

```
def supervisor(query, client):
    dept = classify(query, client)
    answer = specialist(dept, query, client)

    for _ in range(max_revisions):
        review = critic.review(query, answer)
        if review["approved"]:
            return answer

        answer = specialist.revise(
            query, answer, review["feedback"]
        )

    return answer  # best effort after N rounds
```

- → Supervisor controls flow: classify → respond → review → revise
- → Critic feedback is fed back as context for improvement

Shinn et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning." arXiv:2303.11366



Demo — Supervisor-critic pipeline

Switch to terminal: demo/demo.py — Part 2



Debate pattern

Adversarial agents stress-test decisions before commitment

Debate pattern

```
def debate(question, client, rounds=2):
    for r in range(rounds):
        advocate_arg = advocate.argue(question)
        skeptic_arg = skeptic.counter(advocate_arg)

    return judge.decide(
        advocate_arg, skeptic_arg
    )

# Advocate: "Migrating saves 3 sprints..."
# Skeptic:  "Migration risk is too high..."
# Judge:    "The skeptic's risk case wins."
```

- → Multiple rounds sharpen arguments before the judge decides
- → Useful for high-stakes decisions where you want a devil's advocate

Du et al. (2023). "Improving Factuality and Reasoning in Language Models through Multiagent Debate." arXiv:2305.14325



Demo — Structured debate

Switch to terminal: demo/demo.py — Part 3



Shared context + consensus

How agents share data and resolve disagreements

Shared context and tools

- Agents can share a **common tool registry** — same tools, same schemas.
- Or each agent gets a **scoped subset** of tools (principle of least privilege).
- Shared **context object**: conversation so far, retrieved documents, decisions.
- Watch for **race conditions** if agents run in parallel and write to shared state.

Consensus and conflict resolution

- **Voting**: N agents answer independently; majority wins.
- **Ranked choice**: agents score or rank proposals; aggregate.
- **Confidence weighting**: trust the agent that is most sure.
- **Tie-breaking**: supervisor or fallback rule decides deadlocks.
- **Log dissent**: record minority opinions for audit and debugging.

Wang et al. (2022). "Self-Consistency Improves Chain of Thought Reasoning in Language Models." arXiv:2203.11171



Demo — Consensus voting

Switch to terminal: demo/demo.py — Part 4



Swarm + handoffs

Peer-to-peer agents with scoped tools — no central supervisor

Swarm vs supervisor

- **Supervisor**: one agent decides who runs next (router, critic loop).
- **Swarm**: the active agent decides — use a tool, answer, or **hand off**.
- Each agent gets a **scoped tool set** (principle of least privilege).
- **Handoff** is a tool: `transfer_to_engineering` passes full context.

Park et al. (2023). "Generative Agents: Interactive Simulacra of Human Behavior." arXiv:2304.03442

Swarm loop with handoffs

```
def swarm_loop(query, client, start="comms"):
    dept = start
    messages = build_messages(dept, query)
    chain = [dept]

    for hop in range(max_hops):
        msg = run_turn(dept, messages, tools=AGENT_TOOLS[dept])
        if not calls:
            return msg.content, chain

        tool_msgs, transfer = handle_tools(msg)
        messages.extend(tool_msgs)
        if transfer:
            dept = transfer
            chain.append(dept)
        # else: same agent, another tool round

# comms -> engineering -> final answer
```

- → No supervisor — handoff is just another tool call
- → Each department sees only its domain tools + transfer_to_*



Demo — Swarm + tool handoffs

Switch to terminal: demo/demo.py — Part 5



Spawning subagents

Agents created on demand, run in parallel, reports merged back

Route, hand off, or spawn

- **Route**: pick one agent from a roster you wrote. Control returns to you.
- **Hand off**: the active agent passes control sideways and does not get it back.
- **Spawn**: the lead creates a worker on demand, gets a result, keeps control.
- A subagent is a **context window**, not a job title — same model, new conversation.
- Spawn when work is **independent**; hand off when the *domain* changed.

Zhou et al. (2022). "Least-to-Most Prompting Enables Complex Reasoning in Large Language Models." arXiv:2205.10625

Context Mamagement and Speed

- A subagent reads 30 tool results; the lead only ever sees its **summary**.
- The lead stays small, so it stays coherent over a long task.
- Parallelism is useful — three 8s researchers finish in 8s, not 24s.
- The cost: the child cannot see the parent conversation, so the brief must stand alone.

Fan-out with a cap

```
async def fan_out(briefs, client, max_workers=3):
    limit = asyncio.Semaphore(max_workers)

    async def guarded(brief):
        async with limit:
            return await run_subagent(brief, client)

    settled = await asyncio.gather(
        *(guarded(b) for b in briefs),
        return_exceptions=True,      # one failure must
    )                                # not cancel the rest

    return [
        finding if not isinstance(finding, BaseException)
        else stub(brief, finding)    # keep the slots aligned
        for brief, finding in zip(briefs, settled)
    ]
```

- → Module 8 parallelised plan steps with depends_on — same gather, agents instead of steps
- → Semaphore caps concurrency; return_exceptions keeps a crashed worker from taking the batch
- → Return results aligned with the briefs, so finding i still matches tool call i

The subagent is a tool

```
SPAWN_TOOL = {
    "type": "function",
    "name": "spawn_researchers",
    "description": (
        "Research independent sub-questions, each "
        "in its own context. Pass every brief in "
        "one call - they all run at once. Max 3."
    ),
    "parameters": {...},   # briefs: [{department, task}]
}

briefs = to_briefs(spawn_calls(response))  # it wrote
findings = await fan_out(briefs, client)   # you ran

# The output carries the reports only - never
# the children's transcripts.
```

- → The model chooses how many to spawn and what to ask
- → It will validate the agent choice against a dictionary of allowed agents
- → This is the Task tool in Cursor and Claude Code

☑ Reconciling what comes back

- **Always preserve Provenance**: the parent must know where each claim came from.
- **Deterministic merge**: dedupe claims, keep provenance.
- **LLM synthesis**: one merge call — and tell it what the code already flagged.
- **Structured responses** are easier to merge than prose.

⚠ How it goes wrong

- **Overlapping briefs**: two children do the same work and look like corroboration.
- **Unbounded spawning**: children spawning children. Cap depth and total calls.
- **Cost multiplies**, latency does not.
- **Lost provenance**: the parent attaches the source, never the child.
- **Silent drops**: Workers may fail. LLMs are non-deterministic.

Cemri et al. (2025). "Why Do Multi-Agent LLM Systems Fail?" arXiv:2503.13657



Demo — Spawned subagents

Switch to terminal: demo/demo.py — Part 6



Putting it all together

When to use multi-agent — and when not to

Single agent vs multi-agent

Single agent	Multi-agent
Simpler to build and debug	Specialist prompts = better per-domain quality
Lower latency (one LLM call path)	Critic/reviewer catches errors
All context in one conversation	Parallel execution for independent tasks
Sufficient for most use cases	Subagent contexts stay out of the lead
	Higher operational complexity

Summary

One agent until you can justify another

If one prompt can do it, one agent should do it.

Roles stay narrow

Router, specialist, critic, supervisor. Overlap means duplicate work.

Pick one coordination pattern

Supervisor, swarm, debate, or consensus — and stick to it.

Scope tools and log the trace

The critic should not delete production data. Keep the whole conversation.

A subagent is a context window

Spawn for isolation, cap the workers, and never average a conflict away.

Exercises

01 — Router agent: classify queries and dispatch to specialist agents

02 — Supervisor-critic: orchestrate specialists with a quality review loop

03 — Debate + consensus: argue, judge, and vote across multiple agents

04 — Swarm tools: scoped tools and peer-to-peer handoffs between agents

05 — Subagents: spawn researchers in parallel, then reconcile their reports



Module 9 — Complete

Next: frameworks with LangChain